

AutoReduce-BioCRNpyler interface

June 17, 2026

0.1 Create SBML models using BioCRNpyler

```
[ ]: try:
    from biocrnpyler import (
        Enzyme,
        Mixture,
        BasicCatalysis,
        MichaelisMenten,
    )

    default_parameters = {"kb": 100, "ku": 10, "kcat": 1.0}
    E = Enzyme("E", substrates="S", products="P")
    mech_cat = BasicCatalysis()
    default_mechanisms = {mech_cat.mechanism_type: mech_cat}
    M = Mixture(
        "Catalysis Mixture",
        components=[E],
        parameters=default_parameters,
        mechanisms=default_mechanisms,
    )
    CRN = M.compile_crn()
    CRN.write_sbml_file("models/example_1.xml")
except Exception as e:
    print(e)
```

```
[1]: #Also notice that the names of transcript and protein can be changed, or set to
    ↳Species.
    total_RNAP = 10
    total_Ribo = 50
    total_DNA = 0.5
    x0_dict = {"protein_RNAP_machinery":total_RNAP,
               "protein_Ribo_machinery":total_Ribo,
               "dna_X":total_DNA}

    try:
        from biocrnpyler import * # type: ignore
        class GeneExpressionExtract(Mixture):
            def __init__(self, name="", rnape="RNAP", ribosome="Ribo", **kwargs):
```

```

"""Initializes a TxtlExtract instance.

:param name: name of the mixture
:param rnap: name of the RNA polymerase, default: RNAP
:param ribosome: name of the ribosome, default: Ribo
:param kwargs: keywords passed into the parent Class (Mixture)
"""

# Always call the superclass Mixture.__init__(...)
Mixture.__init__(self, name=name, **kwargs)

# create default Components to represent cellular machinery
self.rnap = Protein(rnap, attributes=["machinery"])
self.ribosome = Protein(ribosome, attributes=["machinery"])

default_components = [self.rnap, self.ribosome]
self.add_components(default_components)

# Create default Txtl Mechanisms
mech_tx = Transcription_MM(rnap=self.rnap.get_species())
mech_tl = Translation_MM(ribosome=self.ribosome.get_species())
mech_cat = MichaelisMenten()
mech_bind = One_Step_Binding()

default_mechanisms = {
    mech_tx.mechanism_type: mech_tx,
    mech_tl.mechanism_type: mech_tl,
    mech_cat.mechanism_type: mech_cat,
    mech_bind.mechanism_type: mech_bind
}
self.add_mechanisms(default_mechanisms)
# global mechanisms for dilution and rna degradation
mech_rna_deg_global = Dilution(name="rna_degradation",
                                filter_dict={"rna": True,
                                              "complex": False},
                                default_on=False)
mech_protein_deg_global = Dilution(name="protein_degradation",
                                    filter_dict={"protein": True,
                                                  "machinery": False,
                                                  "complex": False},
                                    default_on=False)
global_mechanisms = {"rna_degradation": mech_rna_deg_global,
                     "protein_degradation": mech_protein_deg_global}
self.add_mechanisms(global_mechanisms)
G = DNAAssembly("X", promoter = "strong", rbs = "weak",
                transcript = None, protein = None)

model_txtl = GeneExpressionExtract(

```

```

    "txt1", components = [G],
    parameter_file = "default_parameters.txt",
    initial_condition_dictionary = x0_dict
)

CRN = model_txt1.compile_crn()
print(repr(CRN), "\n", CRN.pretty_print(show_attributes = False,
                                         show_material = True,
                                         show_rates = True), "\n\n")
CRN.write_sbml_file("models/biocrnpyler_gene_expression.xml")
except Exception as e:
    print(e)
    # print('BioCRNpyler not found. To run this notebook, make sure you run pip
    ↪install biocrnpyler first.')

```

```

Species = dna_X, protein_RNAP_machinery, rna_X,
complex_dna_X_protein_RNAP_machinery_, protein_Ribo_machinery, protein_X,
complex_protein_Ribo_machinery_rna_X_
Reactions = [
    dna[X]+protein[RNAP(machinery)] <--> complex[dna[X]:protein[RNAP]]
    complex[dna[X]:protein[RNAP]] --> dna[X]+rna[X]+protein[RNAP(machinery)]
    rna[X]+protein[Ribo(machinery)] <--> complex[protein[Ribo]:rna[X]]
    complex[protein[Ribo]:rna[X]] -->
rna[X]+protein[X]+protein[Ribo(machinery)]
    rna[X] -->
    protein[X] -->
]
Species(N = 7) = {
complex[protein[Ribo]:rna[X]] (@ 0), complex[dna[X]:protein[RNAP]] (@ 0),
protein[X] (@ 0), rna[X] (@ 0), dna[X] (@ 0), protein[Ribo] (@ 0),
protein[RNAP] (@ 0),
}

Reactions (6) = [
0. dna[X]+protein[RNAP] <--> complex[dna[X]:protein[RNAP]]
Kf=k_forward * dna_X * protein_RNAP_machinery
Kr=k_reverse * complex_dna_X_protein_RNAP_machinery_
k_forward=100.0
found_key=(mech=None, partid=None, name=kb).
search_key=(mech=transcription_mm, partid=strong, name=kb).
k_reverse=0.5
found_key=(mech=None, partid=strong, name=ku).
search_key=(mech=transcription_mm, partid=strong, name=ku).

1. complex[dna[X]:protein[RNAP]] --> dna[X]+rna[X]+protein[RNAP]
Kf=k_forward * complex_dna_X_protein_RNAP_machinery_
k_forward=3.926187672
found_key=(mech=None, partid=strong, name=ktx).

```

```

search_key=(mech=transcription_mm, partid=strong, name=ctx).

2. rna[X]+protein[Ribo] <--> complex[protein[Ribo]:rna[X]]
Kf=k_forward * rna_X * protein_Ribo_machinery
Kr=k_reverse * complex_protein_Ribo_machinery_rna_X_
k_forward=100.0
found_key=(mech=None, partid=None, name=kb).
search_key=(mech=translation_mm, partid=weak, name=kb).
k_reverse=5.0
found_key=(mech=None, partid=weak, name=ku).
search_key=(mech=translation_mm, partid=weak, name=ku).

3. complex[protein[Ribo]:rna[X]] --> rna[X]+protein[X]+protein[Ribo]
Kf=k_forward * complex_protein_Ribo_machinery_rna_X_
k_forward=0.05
found_key=(mech=None, partid=None, name=ctl).
search_key=(mech=translation_mm, partid=weak, name=ctl).

4. rna[X] -->
Kf=k_forward * rna_X
k_forward=0.001
found_key=(mech=None, partid=None, name=kdil).
search_key=(mech=rna_degradation, partid=rna_X, name=kdil).

5. protein[X] -->
Kf=k_forward * protein_X
k_forward=0.001
found_key=(mech=None, partid=None, name=kdil).
search_key=(mech=protein_degradation, partid=protein_X, name=kdil).

]

```

```

c:\Users\ayush\anaconda3\envs\py311-new\Lib\site-
packages\biocrnpyler\parameter.py:507: UserWarning: parameter file contains no
unit column! Please add a column named ['unit', 'units'].
  warn(f"parameter file contains no {accepted_name} column! Please add a "
c:\Users\ayush\anaconda3\envs\py311-new\Lib\site-
packages\biocrnpyler\global_mechanism.py:104: UserWarning: species
complex_protein_Ribo_machinery_rna_X_ has multiple attributes(or material type)
which conflict with global mechanism filter rna_degradation. Using default value
False.
  warn(f"species {repr(s)} has multiple attributes(or material type) which
conflict with global mechanism filter {repr(self)}. Using default value
{self.default_on}.")
c:\Users\ayush\anaconda3\envs\py311-new\Lib\site-
packages\biocrnpyler\global_mechanism.py:104: UserWarning: species

```

protein_RNAP_machinery has multiple attributes(or material type) which conflict with global mechanism filter protein_degradation. Using default value False.

```
warn(f"species {repr(s)} has multiple attributes(or material type) which  
conflict with global mechanism filter {repr(self)}. Using default value  
{self.default_on}.")
```

```
c:\Users\ayush\anaconda3\envs\py311-new\Lib\site-  
packages\biocrnpyler\global_mechanism.py:104: UserWarning: species  
complex_dna_X_protein_RNAP_machinery_ has multiple attributes(or material type)  
which conflict with global mechanism filter protein_degradation. Using default  
value False.
```

```
warn(f"species {repr(s)} has multiple attributes(or material type) which  
conflict with global mechanism filter {repr(self)}. Using default value  
{self.default_on}.")
```

```
c:\Users\ayush\anaconda3\envs\py311-new\Lib\site-  
packages\biocrnpyler\global_mechanism.py:104: UserWarning: species  
protein_Ribo_machinery has multiple attributes(or material type) which conflict  
with global mechanism filter protein_degradation. Using default value False.
```

```
warn(f"species {repr(s)} has multiple attributes(or material type) which  
conflict with global mechanism filter {repr(self)}. Using default value  
{self.default_on}.")
```

```
c:\Users\ayush\anaconda3\envs\py311-new\Lib\site-  
packages\biocrnpyler\global_mechanism.py:104: UserWarning: species  
complex_protein_Ribo_machinery_rna_X_ has multiple attributes(or material type)  
which conflict with global mechanism filter protein_degradation. Using default  
value False.
```

```
warn(f"species {repr(s)} has multiple attributes(or material type) which  
conflict with global mechanism filter {repr(self)}. Using default value  
{self.default_on}.")
```

```
[2]: from IPython.core.interactiveshell import InteractiveShell  
InteractiveShell.ast_node_interactivity = "all"  
  
from autoreduce import *  
import numpy as np  
from sympy import symbols
```

```
[3]: from autoreduce.converters import load_sbml  
from sympy import Symbol  
sys = load_sbml('models/biocrnpyler_gene_expression.xml',  
               outputs = ['protein_X'])  
for x, x_ic in x0_dict.items():  
    curr_ind = sys.x.index(Symbol(x))  
    sys.x_init[curr_ind] = x_ic
```

```
[4]: sys.x
```

```
[4]: [dna_X,
      protein_RNAP_machinery,
      rna_X,
      complex_dna_X_protein_RNAP_machinery_,
      protein_Ribo_machinery,
      protein_X,
      complex_protein_Ribo_machinery_rna_X_]

[5]: new_params_values = [100.0, 0.5, 3.926187672,
                          5.0, 0.05, 0.01]
     sys.params_values = new_params_values
```

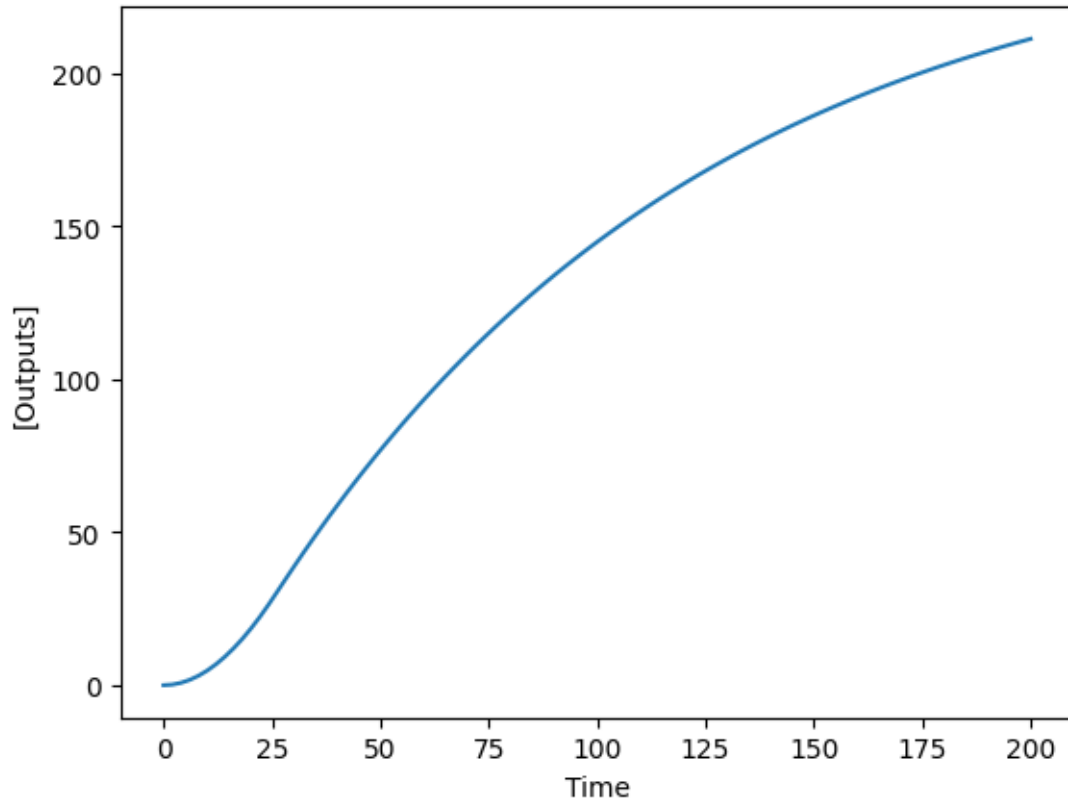
1 Create and solve the ODE for the model

```
[6]: from autoreduce.utils import get_ODE
     import numpy as np
     timepoints_ode = np.linspace(0, 200, 100)
     sys_ode = get_ODE(sys, timepoints_ode)
     sol = sys_ode.solve_system().T
     try:
         import matplotlib.pyplot as plt
         full_model = np.transpose(np.array(sys.C)@sol)
         plt.plot(timepoints_ode, full_model)
         plt.xlabel('Time')
         plt.ylabel('[Outputs]')
         plt.show()
     except:
         print('Plotting libraries missing.')
```

```
[6]: [<matplotlib.lines.Line2D at 0x293e7fa7c10>]
```

```
[6]: Text(0.5, 0, 'Time')
```

```
[6]: Text(0, 0.5, '[Outputs]')
```



2 Local sensitivity analysis for BioCRNpyler model

```
[7]: from autoreduce.utils import get_SSM
timepoints_ssm = np.linspace(0,20,10)
sys_ssm = get_SSM(sys, timepoints_ssm)
# Uncomment to run
# Ss = sys_ssm.compute_SSM() # len(timepoints) x len(params) x len(states)

[8]: # out_Ss = []
# for i in range(len(sys.params)):
#     out_Ss.append((np.array(sys.C)@(Ss[:,i,:].T)))
# out_Ss = np.reshape(np.array(out_Ss), (len(timepoints_ssm), len(sys.params), 1))
# ↪ 1))
```

3 Plot heatmap for sensitivity analysis

```
[9]: # try:
#     import seaborn as sn
#     import matplotlib.pyplot as plt
```

```
#     sn.heatmap(out_Ss[:, :, 0].T)
#     plt.xlabel('Time')
#     plt.ylabel('Parameters')
#     plt.title('Sensitivity of protein with all parameters'.format(j))
#     plt.show()
# except:
#     print('Plotting libraries missing.')
```

4 Model reduction starts here:

```
[10]: sys = load_sbml('models/biocrnpyler_gene_expression.xml',
                    outputs = ['protein_X'])
for x, x_ic in x0_dict.items():
    curr_ind = sys.x.index(Symbol(x))
    sys.x_init[curr_ind] = x_ic
```

```
[11]: sys.x
```

```
[11]: [dna_X,
       protein_RNAP_machinery,
       rna_X,
       complex_dna_X_protein_RNAP_machinery_,
       protein_Ribo_machinery,
       protein_X,
       complex_protein_Ribo_machinery_rna_X_]
```

5 Conservation Laws (if any)

```
[12]: G, P, T, C1, R, X, C2 = sys.x
conserved_sets = [[P,C1],[R,C2]]
con = sys.solve_conservation_laws(conserved_sets=conserved_sets,
                                states_to_eliminate=[P, R])
```

Found conservation laws: [complex_dna_X_protein_RNAP_machinery_ +
protein_RNAP_machinery - protein_RNAP_machinery_total,
complex_protein_Ribo_machinery_rna_X_ + protein_Ribo_machinery -
protein_Ribo_machinery_total]

```
[13]: sys.params_values = new_params_values + [total_RNAP, total_Ribo]
```

```
[14]: sys.x
```

```
[14]: [dna_X,
       rna_X,
       complex_dna_X_protein_RNAP_machinery_,
       protein_X,
```



```
complex_protein_Ribo_machinery_rna_X_]
```

```
[15]: from autoreduce.utils import get_ODE
timepoints_ode = np.linspace(0, 200, 100)
sys_ode = get_ODE(sys, timepoints_ode)
sol = sys_ode.solve_system().T
try:
    import matplotlib.pyplot as plt
    plt.plot(timepoints_ode, np.transpose(np.array(sys.C)@sol),
             lw = 3, label = 'Reduced model')
    plt.plot(timepoints_ode, full_model, '--',
             alpha = 0.5, lw = 5, label = 'Full model')
    plt.xlabel('Time')
    plt.ylabel('[Outputs]')
    plt.legend()
    plt.show()
except:
    print('Plotting libraries missing.')
```

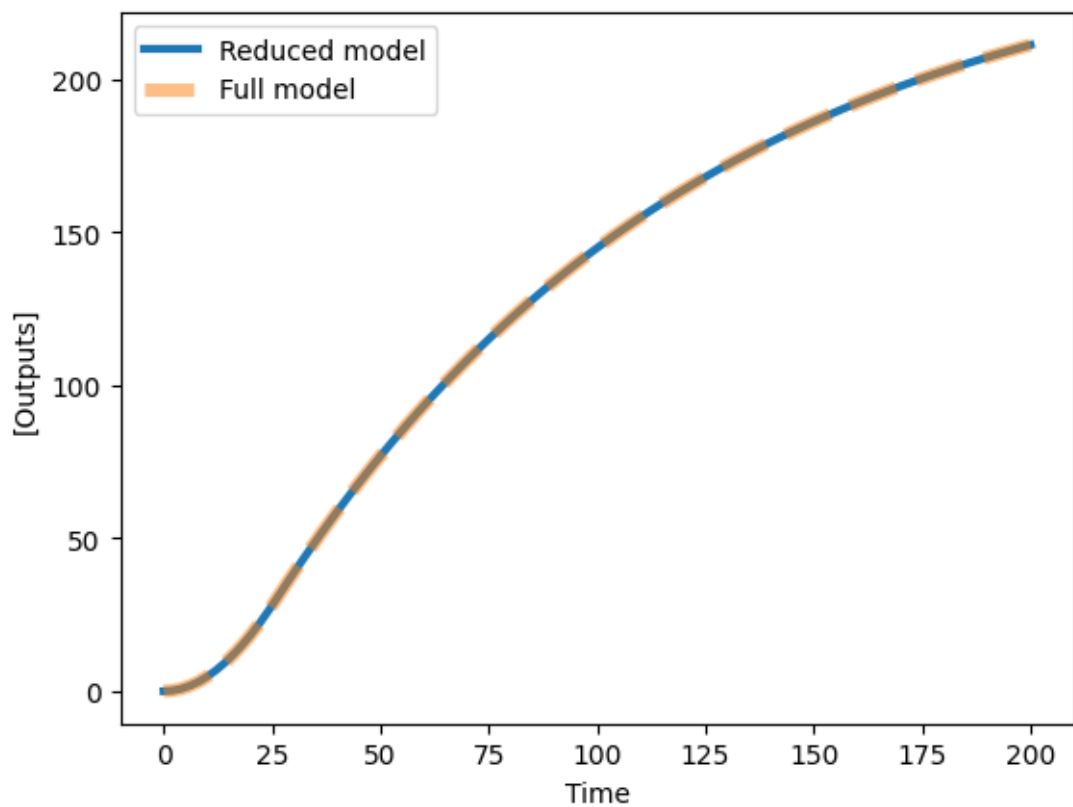
```
[15]: [<matplotlib.lines.Line2D at 0x293ffc16fd0>]
```

```
[15]: [<matplotlib.lines.Line2D at 0x293ffc16550>]
```

```
[15]: Text(0.5, 0, 'Time')
```

```
[15]: Text(0, 0.5, '[Outputs]')
```

```
[15]: <matplotlib.legend.Legend at 0x293ffc27d10>
```



```
[16]: sys.x
```

```
[16]: [dna_X,
      rna_X,
      complex_dna_X_protein_RNAP_machinery_,
      protein_X,
      complex_protein_Ribo_machinery_rna_X]
```

6 Derive model with no intermediate complexes

```
[17]: reduced_sys, fast_ss = sys.solve_timescale_separation([G,T,X])
```

Successful solution obtained with states: [dna_X, rna_X, protein_X]!

```
[18]: reduced_sys.x
```

```
[18]: [dna_X, rna_X, protein_X]
```

```
[19]: from autoreduce.utils import get_ODE
      timepoints_ode = np.linspace(0, 200, 100)
      # reduced_sys.params_values = new_params_values + []
```

```

reduced_ode = get_ODE(reduced_sys, timepoints_ode)
sol = reduced_ode.solve_system().T
try:
    import matplotlib.pyplot as plt
    plt.plot(timepoints_ode, sol[2,:], lw = 3, label='Reduced Model')
    plt.plot(timepoints_ode, full_model, '--', lw = 5, alpha = 0.5, label='Full
    Model')
    plt.xlabel('Time')
    plt.ylabel('[Outputs]')
    plt.legend()
    plt.show()
except:
    print('Plotting libraries missing.')

```

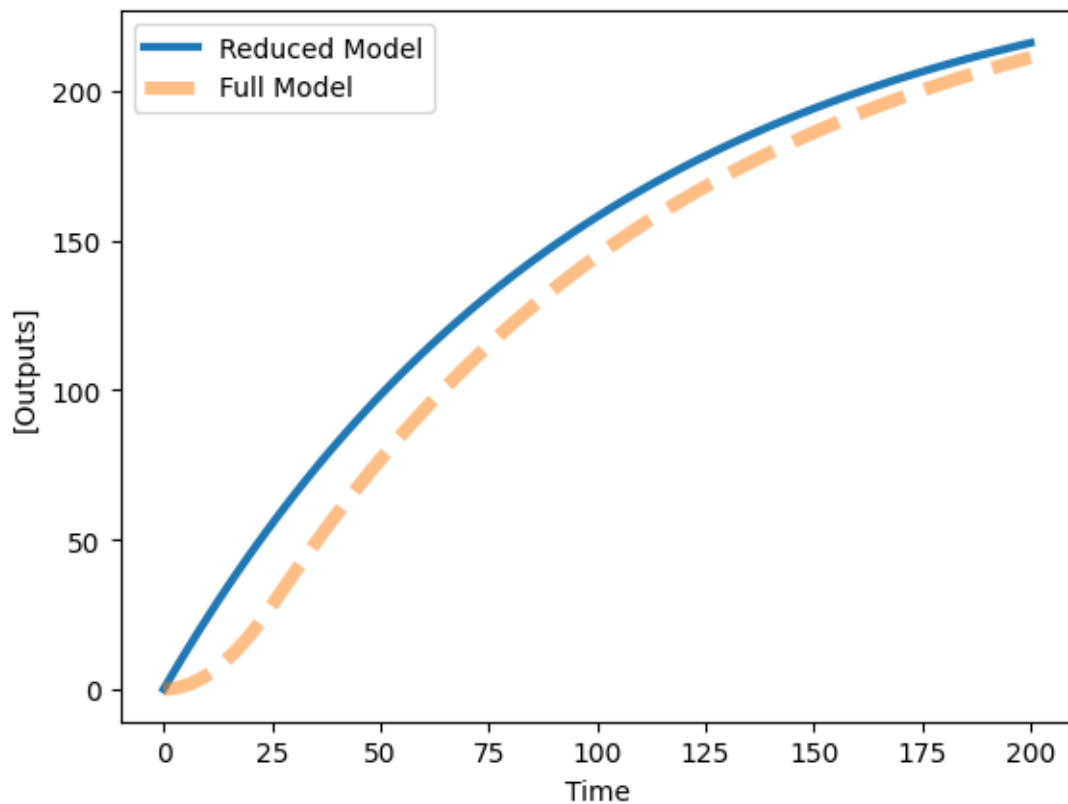
[19]: [<matplotlib.lines.Line2D at 0x293ffd4c310>]

[19]: [<matplotlib.lines.Line2D at 0x293ffbf9310>]

[19]: Text(0.5, 0, 'Time')

[19]: Text(0, 0.5, '[Outputs]')

[19]: <matplotlib.legend.Legend at 0x293ffd36f50>



6.1 Write the reduced model to an SBML file to integrate other tools!

```
[20]: reduced_sys.write_sbml("models/reduced_gene_expression.xml")
```

```
[20]: <SBMLDocument>
```